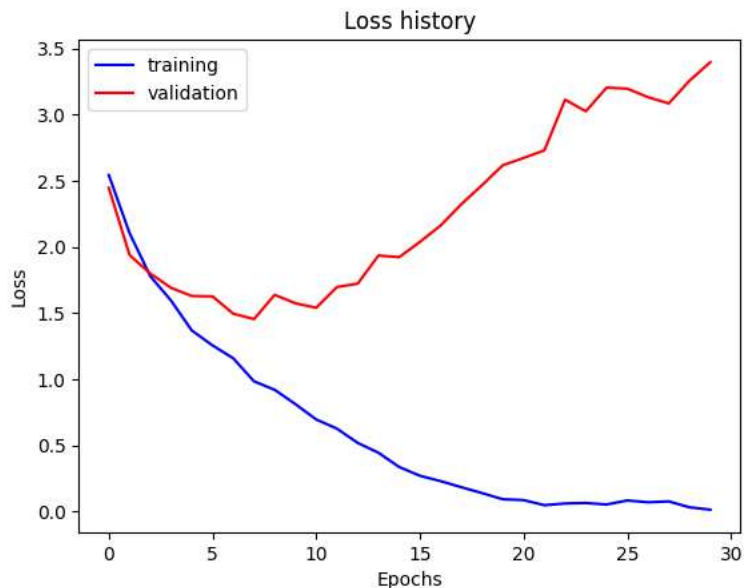


Part 1: SimpleNet

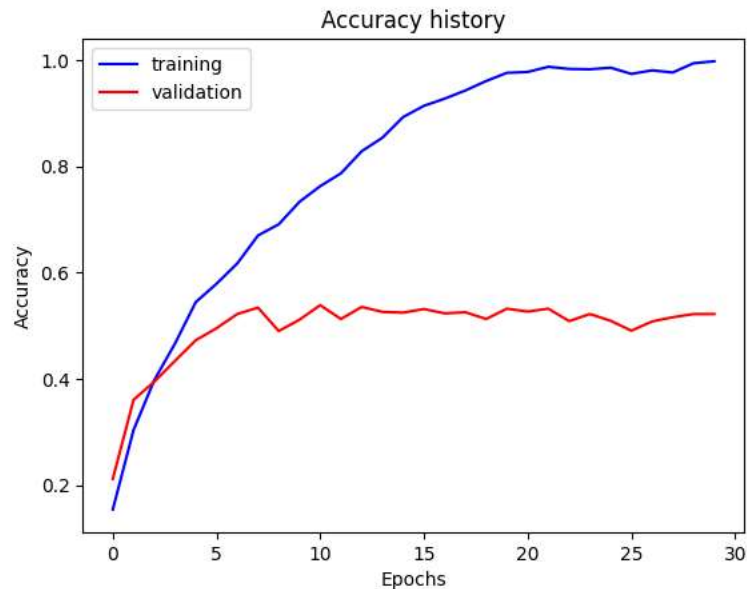
Insert loss plot for SimpleNet here



Final training accuracy: 0.997319932998325

Final validation accuracy: 0.522

Insert accuracy plot for SimpleNet here



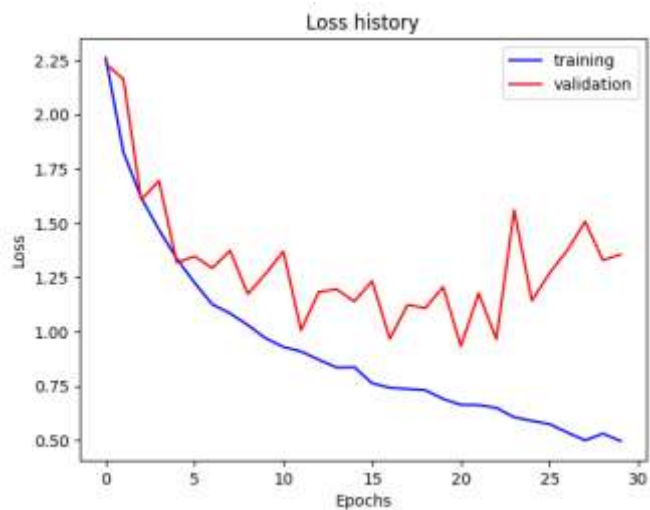
Part 2: SimpleNetFinal

Add each of the following (keeping the changes as you move to the next row):

	Training accuracy	Validation accuracy
SimpleNet	0.6268	0.171
+ Jittering	0.4917	0.315
+ Zero-centering & variance-normalization	0.5632	0.348
+ Dropout regularization	0.5410	0.412
+ Making network "deep"	0.7024	0.525
+ Batch normalization	0.827	0.608

Part 2: SimpleNetFinal

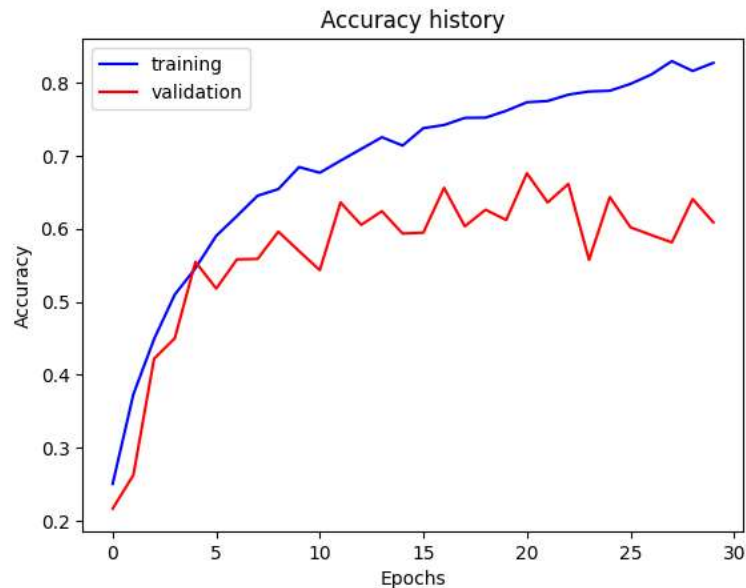
Insert loss plot for SimpleNetFinal here:



Final training accuracy: 0.8271356783919598

Final validation accuracy: 0.608667

Insert accuracy plot for SimpleNetFinal here



Part 2: SimpleNetFinal

[Name 10 different possible transformations for data augmentation.]

Horizontal flip, vertical flip, random rotation, random cropping, color jitter, gaussian noise addition, random scaling/resizing, random translation, random perspective distortion, cutout/random erasing.

[What is the desired variance after each layer? Why would that be helpful?]

The desired variance is around 1. Having unit variance prevents activations from being too large/small as they go through the network. If the variance is too large, activations can explode and make training unstable, and if its too small, gradients can vanish and slow the learning. Keeping the variance near 1 stabilizes training and improves gradient flow through the network.

Part 2: SimpleNetFinal

[What distribution is dropout usually sampled from?]

Dropout is sampled from a Bernoulli distribution. Each neuron is independently kept with probability p or dropped, with probability $1-p$. This prevents the network from relying too much on specific neurons and helps reduce overfitting.

[How many parameters does your base SimpleNet model have? How many parameters does your SimpleNetFinal model have?]

The SimpleNet model has 549647 parameters, and the SimpleNetFinal model has 423631 parameters. The total number of parameters can decrease depending on how the architecture is modified, and batch normalization introduces a small number of parameters. Restructuring connected layers can reduce the total parameter count while improving performance.

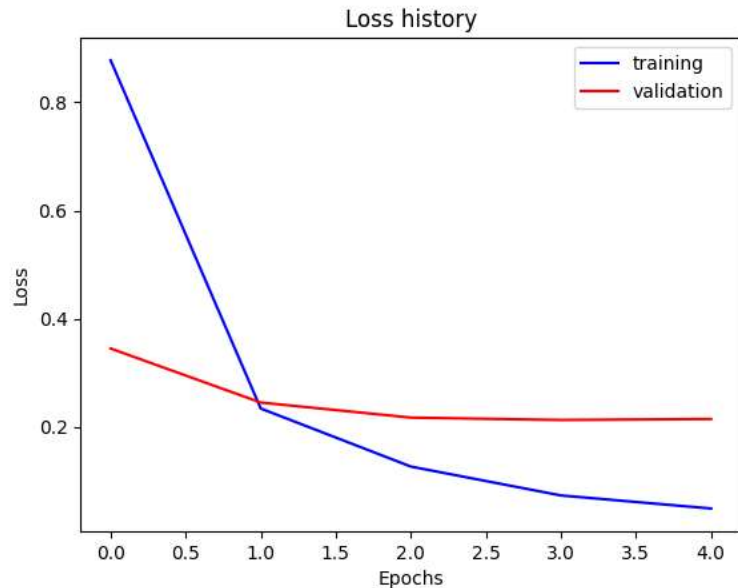
What is the effect of batch norm after a conv layer with a bias?

Batch normalizations normalizes the activations of a layer by subtracting the batch mean and dividing by the batch standard deviation.

Because this normalization removes the mean, the bias term of the convolution layer is redundant, making the bias parameter have little effect and can be removed without impacting the performance.

Part 3: ResNet

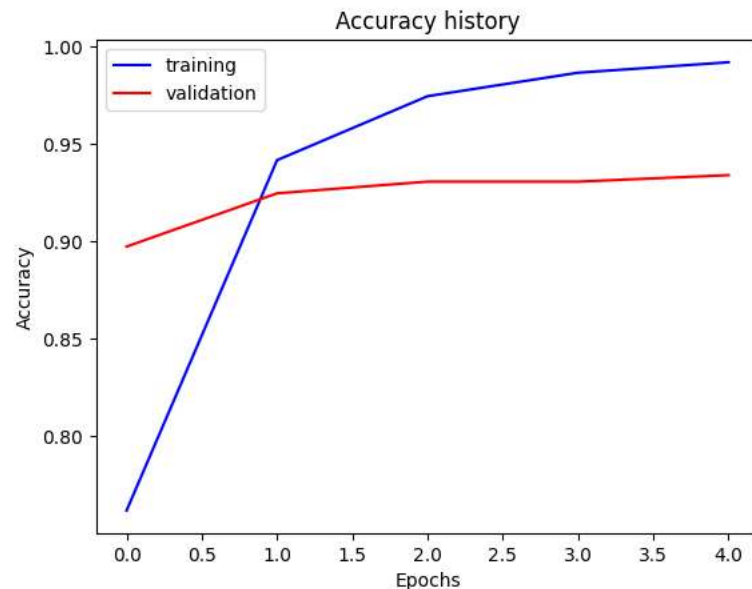
Insert loss plot here



Final training accuracy: 0.9919597989949749

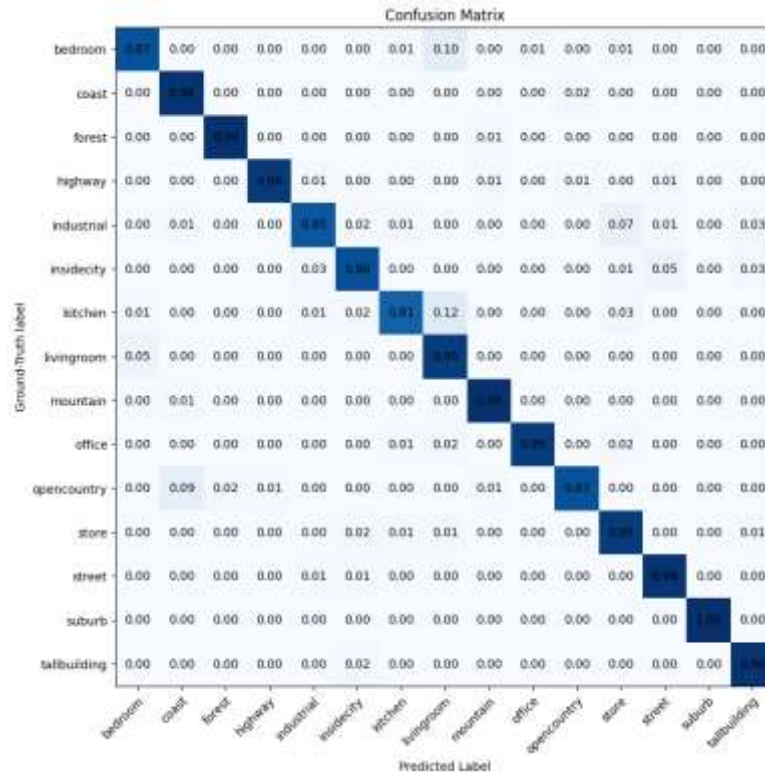
Final validation accuracy: 0.934

Insert accuracy plot here



Part 3: ResNet

Insert visualization of confusion matrix obtained from your final ResNet model.



Part 3: ResNet

Insert visualizations of 3 misclassified images from the most misclassified class according to your confusion matrix. Explain why this may have occurred.

True: livingroom → Misclassified as: store



True: bedroom → Misclassified as: store



True: office → Misclassified as: store



All three misclassified pairs have indoor scenes being predicted as “store”. Living rooms, offices, and bedrooms share visual features with stores – shelves, counters, furniture, and structured indoor layouts. Since the model relies on texture and spatial patterns, these overlapping features cause it to confuse these categories. Additionally, ‘store’ is a visually diverse class that can resemble many indoor environments.

Part 3: ResNet

What does fine-tuning a network mean?

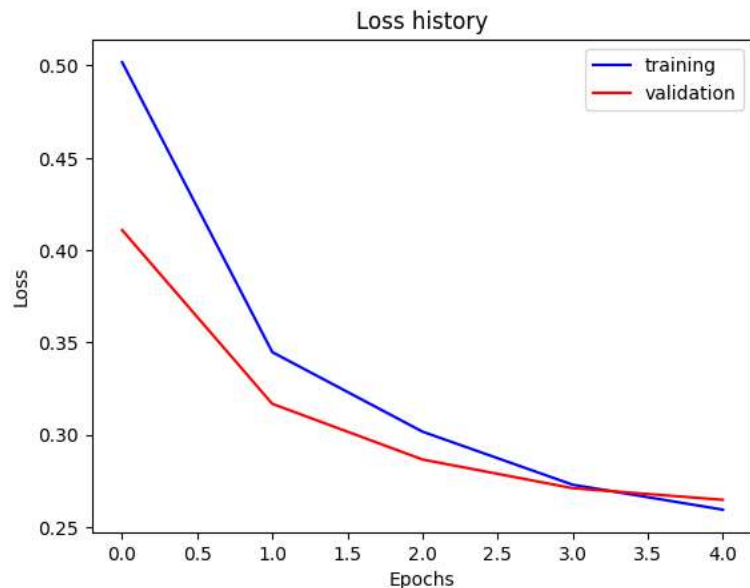
Fine tuning means taking a pretrained neural network and continuing to train it on a new dataset. Instead of training it from scratch, the model starts with weights learned from a large dataset and adjusts it to better fit the new task.

Why do we want to "freeze" the conv layers and some of the linear layers from a pre-trained ResNet? Why can we do this?

We freeze the conv layers since they already learn important low level features like edges or textures from large datasets. These features are general and transferable across vision tasks. By freezing them, we reduce the training time, need less data and prevent overfitting.

Part 4: Multi-label Scene Attributes (Extra Credit)

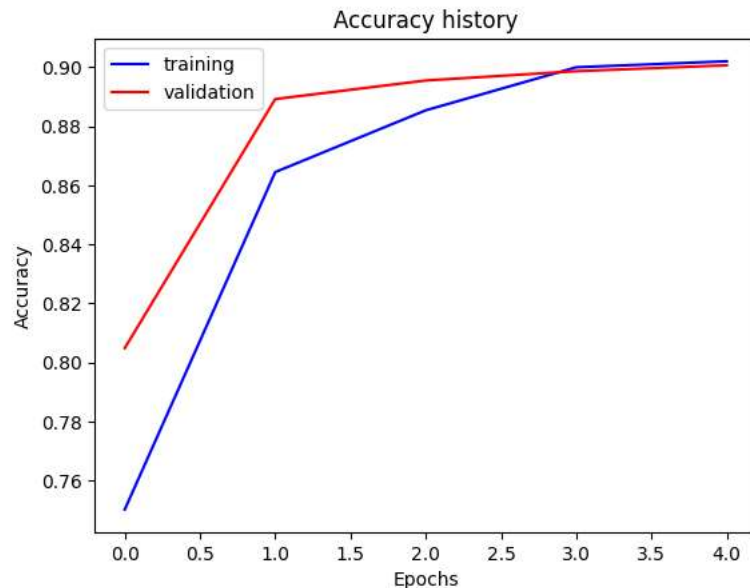
Insert loss plot here:



Final training accuracy: 0.901935021500239

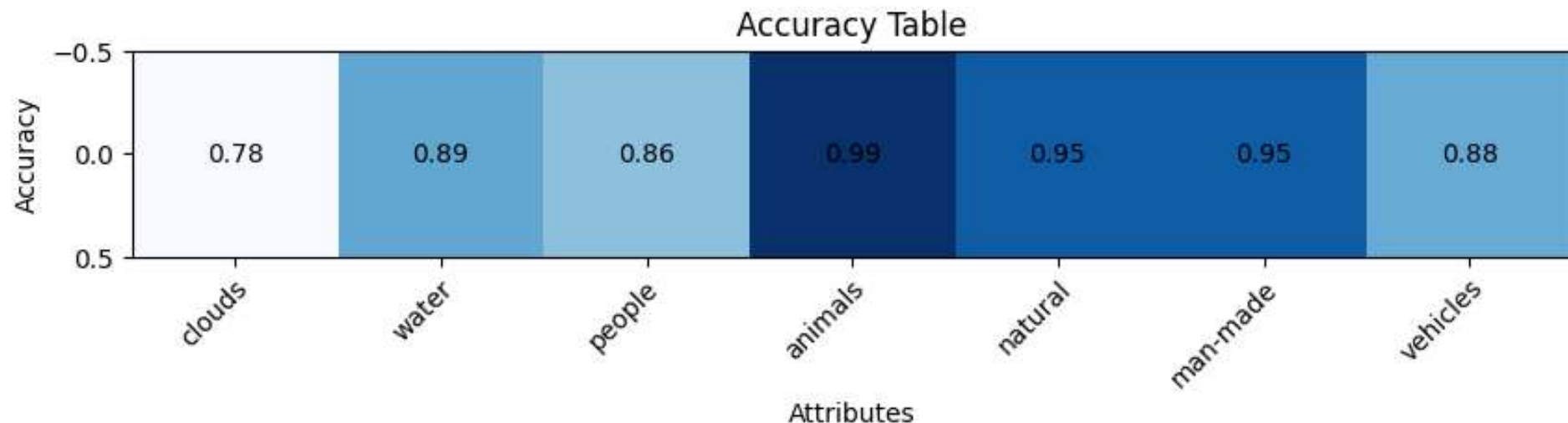
Final validation accuracy: 0.90057142857143

Insert accuracy plot here



Part 4: Multi-label Scene Attributes (Extra Credit)

Insert visualization of accuracy table obtained from your final MultilabelResNet model.



Part 4: Multi-label Scene Attributes (Extra Credit)

List 3 changes that you made in the network compared to the one in part 3.

1. The output layer was modified to make multiple attribute predictions instead of a single class label.
2. The loss function was changed from `CrossEntropyLoss` to `BCEWithLogitsLoss`
3. `BCEWithLogitsLoss` adds sigmoid internally to each output, unlike softmax in part 3 that normalizes outputs across all classes, which lets multiple attributes to be predicted simultaneously.

[Is the loss function of the ResNet model from part 3 appropriate for this problem? Why or why not?]

No, `CrossEntropyLoss` assumes that each input belongs to one class, which is suitable for single label classification problems. In multilabel classification, an image can have multiple attributes. Therefore, `BCEWithLogitsLoss` is more appropriate since it treats each label prediction by itself.

Part 4: Multi-label Scene Attributes (Extra Credit)

[Explain a problem that one needs to be wary of with multilabel classification. HINT: consider the purpose of visualizing your results with the accuracy table. You might want to do some data exploration here.]

One challenge in multilabel classification is label imbalance. Some attributes appear more frequently than others in the dataset. Because of that, the model might be biased towards predicting common attributes while failing to detect rare ones. The accuracy table reveals this problem because some attributes might show lower accuracy than others.